

Applying First Normal Form (1NF) and Second Normal Form (2NF)

1. Objective

In Week 4 the design was analysed and redundancy, partial dependencies and anomalies were found. The objective of Week 5 is to:

1. Apply **First Normal Form (1NF)** to remove repeating groups and non-atomic values.
2. Apply **Second Normal Form (2NF)** to remove partial dependencies.
3. Modify the database schema wherever necessary and **document every change made**.

2. Recap of Weeks 1 to 4

- **Weeks 1 to 3:** Business requirements were studied. Entities Customer, Category, Product, Supplier, Orders, Order_Details, Payment, Shipment and Review were identified with their primary keys, relationships were defined and a relational schema was prepared.
- **Week 4:** The single-table design **ORDER_FLAT** was analysed. Its primary key is (Order_ID, Product_ID) and these dependencies were found:

Type	Dependency
Full	(Order_ID, Product_ID) → Quantity, Unit_Price
Partial	Order_ID → Order_Date, Order_Status, Customer_ID, Payment_Method, Payment_Status, Shipment_Status
Partial	Product_ID → Product_Name, Price, Category_ID, Supplier_ID
Transitive	Customer_ID → Cust_Name, Cust_Email, Cust_Mobile, Cust_Address
Transitive	Category_ID → Category_Name
Transitive	Supplier_ID → Supplier_Name

A design issue was also noted: Supplier and Product are linked many-to-many, so a Product_Supplier table is needed.

3. Normalization: Basic Idea

Normal form	Rule
1NF	Every column holds a single (atomic) value, there are no repeating groups, and every row is

Normal form	Rule
	uniquely identified by a primary key
2NF	The table is in 1NF and no non-key attribute depends on only <i>part</i> of a composite primary key (no partial dependency)

Note: 2NF only matters for tables with a composite primary key. A table whose primary key is a single column is automatically in 2NF if it is in 1NF.

4. First Normal Form (1NF)

4.1 Unnormalized Table (UNF)

A raw order record, as a shop might first write it, stores all products of an order in one row:

Order_ID	Customer_ID	Cust_Name	Cust_Address	Product_IDs	Product_Names	Quantities
1001	C01	Arun	12 Anna Nagar, Trichy, Tamil Nadu, 620017	P01, P02	Laptop, Mouse	1, 2
1002	C01	Arun	12 Anna Nagar, Trichy, Tamil Nadu, 620017	P02	Mouse	1
1003	C02	Divya	4 Gandhi Rd, Chennai, Tamil Nadu, 600001	P01	Laptop	1

4.2 1NF Violations Found

No.	Violation	Where
1	Repeating group / multi-valued cells: several values in one cell	Product_IDs, Product_Names, Quantities in order 1001
2	Non-atomic (composite) attribute: street, city, state and pincode in one column	Cust_Address (and Delivery_Address)

No.	Violation	Where
3	No suitable primary key: Order_ID alone cannot identify a product line	Whole table

4.3 Steps Taken to Reach 1NF

1. **Remove the repeating group.** Create one row for each product in an order.
2. **Make the address atomic.** Split Address into Street, City, State, Pincode, so that customers can be searched by city or pincode.
3. **Define the primary key.** Because one order has many products, the key becomes the composite (Order_ID, Product_ID).

4.4 Result: ORDER_FLAT in 1NF

Order_ID	Product_ID	Customer_ID	Cust_Name	Street	City	State	Pincode	Product_Name	Price	Quantity
1001	P01	C01	Arun	12 Anna Nagar	Trichy	Tamil Nadu	620017	Laptop	55000	1
1001	P02	C01	Arun	12 Anna Nagar	Trichy	Tamil Nadu	620017	Mouse	500	2
1002	P02	C01	Arun	12 Anna Nagar	Trichy	Tamil Nadu	620017	Mouse	500	1
1003	P01	C02	Divya	4 Gandhi Rd	Chennai	Tamil Nadu	600001	Laptop	55000	1

1NF schema:

ORDER_FLAT (Order_ID, Product_ID, Order_Date, Order_Status, Customer_ID, Cust_Name, Cust_Email, Cust_Mobile, Street, City, State, Pincode, Product_Name, Price, Category_ID, Category_Name, Supplier_ID, Supplier_Name, Quantity, Unit_Price, Payment_Method, Payment_Status, Shipment_Status)

Primary key: (Order_ID, Product_ID). All values are now atomic and there are no repeating groups, so the table is in **1NF**. Redundancy is still high, which is addressed next.

5. Second Normal Form (2NF)

5.1 Check for Partial Dependencies

The key is composite (Order_ID, Product_ID), so each non-key attribute is tested against the whole key:

Attribute(s)	Depends on	Result
Quantity, Unit_Price	Order_ID and Product_ID	Full dependency, stays
Order_Date, Order_Status, Customer_ID, Cust_Name, Cust_Email, Cust_Mobile, address fields, Payment_Method, Payment_Status, Shipment_Status	Order_ID only	Partial dependency
Product_Name, Price, Category_ID, Category_Name, Supplier_ID, Supplier_Name	Product_ID only	Partial dependency

Order-level facts repeat for every product in the order, and product-level facts repeat for every order of the product. This is the cause of the Week 4 anomalies.

5.2 Steps Taken to Reach 2NF

1. For each partial dependency, move the dependent attributes into a **new table** whose primary key is the part of the key they depend on.
2. Keep the determinant as the primary key of the new table, and leave a copy in the original table as a **foreign key**.
3. Leave only fully dependent attributes in the original table.

5.3 Decomposition

New table	Primary key	Attributes moved
ORDERS	Order_ID	Order_Date, Order_Status, Customer_ID, Cust_Name, Cust_Email, Cust_Mobile, Street, City, State, Pincode, Payment_Method, Payment_Status, Shipment_Status
PRODUCT	Product_ID	Product_Name, Price, Category_ID, Category_Name, Supplier_ID, Supplier_Name

New table	Primary key	Attributes moved
ORDER_DETAILS	(Order_ID, Product_ID)	Quantity, Unit_Price

5.4 Result: Schema in 2NF

ORDERS (Order_ID, Order_Date, Order_Status, Customer_ID, Cust_Name, Cust_Email, Cust_Mobile, Street, City, State, Pincode, Payment_Method, Payment_Status, Shipment_Status)

PRODUCT (Product_ID, Product_Name, Price, Category_ID, Category_Name, Supplier_ID, Supplier_Name)

ORDER_DETAILS (Order_ID, Product_ID, Quantity, Unit_Price) Foreign keys: Order_ID → ORDERS, Product_ID → PRODUCT

5.5 Sample Data After 2NF

ORDERS

Order_ID	Order_Date	Customer_ID	Cust_Name	City
1001	2026-09-01	C01	Arun	Trichy
1002	2026-09-05	C01	Arun	Trichy
1003	2026-09-07	C02	Divya	Chennai

PRODUCT

Product_ID	Product_Name	Price	Category_ID	Category_Name
P01	Laptop	55000	CAT1	Electronics
P02	Mouse	500	CAT1	Electronics

ORDER_DETAILS

Order_ID	Product_ID	Quantity	Unit_Price
1001	P01	1	55000
1001	P02	2	500
1002	P02	1	500
1003	P01	1	55000

The Laptop's name and price are now stored once. Unit_Price stays in ORDER_DETAILS because it records the price paid at the time of purchase.

6. Applying 1NF and 2NF to the Entity Schema

The same checks are applied to the entity tables designed in Weeks 2 and 3, including the new Product_Supplier table from Week 4.

Table	Primary key	1NF check	2NF check
Customer	Customer_ID	Address split into Street, City, State, Pincode (changed)	Single-column key, satisfied
Category	Category_ID	Atomic	Single-column key, satisfied
Supplier	Supplier_ID	Contact_Info kept as one contact value; if it holds both phone and email, split into Phone and Email (changed)	Single-column key, satisfied
Product	Product_ID	Atomic	Single-column key, satisfied
Orders	Order_ID	Atomic	Single-column key, satisfied
Order_Details	Order_Detail_ID	Atomic	Single-column key, satisfied. Also (Order_ID, Product_ID) → Quantity, Unit_Price is a full dependency
Payment	Payment_ID	Atomic	Single-column key, satisfied
Shipment	Shipment_ID	Delivery_Address split into Street, City, State, Pincode (changed)	Single-column key, satisfied
Review	Review_ID	Atomic	Single-column key, satisfied
Product_Supplier	(Product_ID, Supplier_ID)	Atomic	Supply_Price depends on both Product_ID and Supplier_ID, so there is no partial dependency

7. Documentation of Changes Made

No.	Change	Reason	Normal form
1	Product list in one order split into one row per product	Remove repeating group	1NF
2	Address split into Street, City, State, Pincode (Customer and Shipment)	Make attribute atomic	1NF
3	Composite key (Order_ID, Product_ID) defined	Uniquely identify each order line	1NF
4	Order-level attributes moved to ORDERS	Remove partial dependency on Order_ID	2NF
5	Product-level attributes moved to PRODUCT	Remove partial dependency on Product_ID	2NF
6	ORDER_DETAILS keeps only Quantity and Unit_Price	Keep only fully dependent attributes	2NF
7	Foreign keys Order_ID and Product_ID added to ORDER_DETAILS	Link the new tables	2NF

8. Effect on Redundancy and Anomalies

Week 4 problem	Status after 2NF
Product cannot be added until it is ordered (insertion)	Fixed: PRODUCT is a separate table
Deleting the only order of a product removes the product (deletion)	Fixed: product rows are independent of orders
Product price stored on every order line (update)	Fixed: stored once in PRODUCT
Order date and status repeated per line item	Fixed: stored once in ORDERS
Customer cannot be registered without an order (insertion)	Still present: customer details are inside ORDERS

Week 4 problem	Status after 2NF
Customer address repeated in every order of that customer (update)	Still present
Category name repeated for every product in the category	Still present: Category_Name is inside PRODUCT
Supplier name repeated for every product of the supplier	Still present: Supplier_Name is inside PRODUCT

The remaining problems are caused by the transitive dependencies identified in Week 4 (Customer_ID → customer details, Category_ID → Category_Name, Supplier_ID → Supplier_Name). 1NF and 2NF do not remove them, so the design needs further improvement.

9. Conclusion

The single-table design was first brought to 1NF by removing repeating groups, making addresses atomic and defining a composite primary key. It was then brought to 2NF by splitting it into ORDERS, PRODUCT and ORDER_DETAILS, which removed all partial dependencies and fixed several insertion, deletion and update anomalies. The entity tables from Weeks 2 and 3 were also checked, with the address fields split for atomicity. Some redundancy caused by transitive dependencies remains and will need to be addressed to complete the design improvement.